Phát hiện botnet IoT bằng autoencoder nhẹ

Đề tài DT043 · Bài giữa kỳ An toàn Bảo mật Hệ thống thông tin

Tên: Hoàng Thanh Trà

MSSV: 1150080120

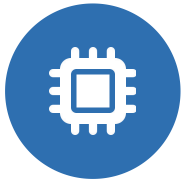
Lớp: 11CNPM2

Bài toán: phát hiện botnet ngay tại edge



Thiết bị IoT dễ bị chiếm

Mật khẩu mặc định, ít vá – Mirai, BASHLITE chiếm hàng loạt để DDoS



Edge có tài nguyên hạn chế

Bộ phát hiện phải nhỏ (KB), nhanh (µs), ít RAM và năng lượng



Mỗi thiết bị một hành vi

Học benign riêng từng thiết bị, cảnh báo khi lỗi tái tạo tăng

Câu hỏi nghiên cứu

- RQ1 Tái lập baseline Meidan 2018
- RQ2 Per-device có hơn model chung?
- RQ3 AE nhẹ giữ chất lượng?
- RQ4 int8 + hiệu chỉnh ngưỡng
- RQ5 Chạy được trên edge?
- RQ6 AE kích hoạt MTD lai

Tiêu chí thành công chốt trước khi chạy (SHA-256 ghi trong nhật ký)

Bài toán mà bài báo đặt ra

Bối cảnh

Tấn công mạng nhắm vào hạ tầng và thiết bị IoT ngày càng nhiều. Moving Target Defence (MTD) liên tục thay đổi bề mặt tấn công – địa chỉ, hệ điều hành, số bản sao máy – để kẻ tấn công khó khai thác và tốn công hơn.

Vấn đề cốt lõi

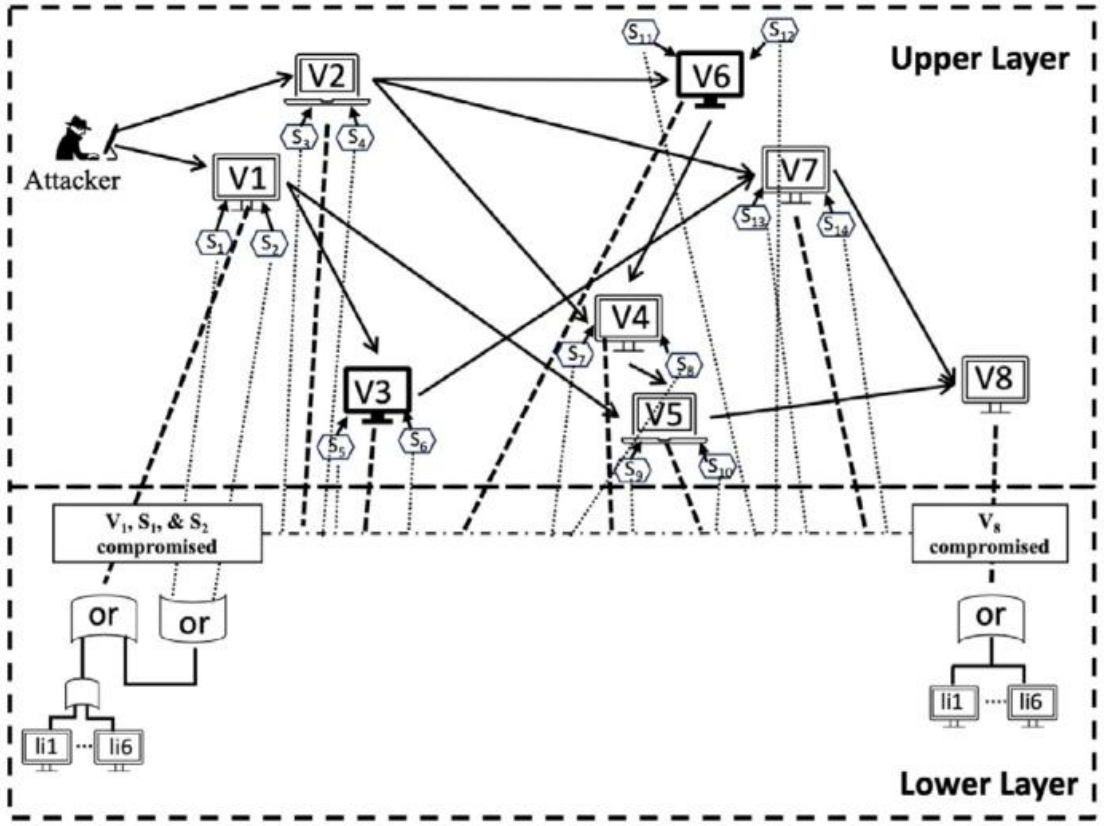
Chưa có đánh giá hiệu quả khi kết hợp nhiều kỹ thuật MTD theo thời gian, chi phí và công sức tấn công. IoT không đồng nhất, tài nguyên hạn chế, nên MTD có thể tiêu tốn quá mức tài nguyên mạng và hệ thống.

Yêu cầu đặt ra

Cần phương pháp định lượng mức an toàn của từng trạng thái mạng sau mỗi lần MTD, so sánh shuffle, diversity, redundancy và tổ hợp của chúng để chọn chiến lược phòng thủ tốt nhất.

Masud, Keshk, Moustafa, Turnbull, Susilo – Vulnerability defence using hybrid moving target defence in Internet of Things systems. Computers & Security 153 (2025) 104380 · DOI 10.1016/j.cose.2025.104380 · Open Access (CC BY)

Phương pháp: mô hình 3-layer THARM và ba kỹ thuật MTD



Hình 3 của bài báo: tầng trên (máy ảo, dịch vụ) và tầng dưới (cây lỗ hổng OR)

Mô hình TH = (N, U, S, L, C)

- N: các trạng thái mạng theo thời gian
- U: đồ thị tấn công giữa 8 máy ảo
- S: đồ thị tấn công giữa 14 dịch vụ
- L: cây tấn công (cổng OR) gồm lỗ hổng OS + dịch vụ của từng máy ảo

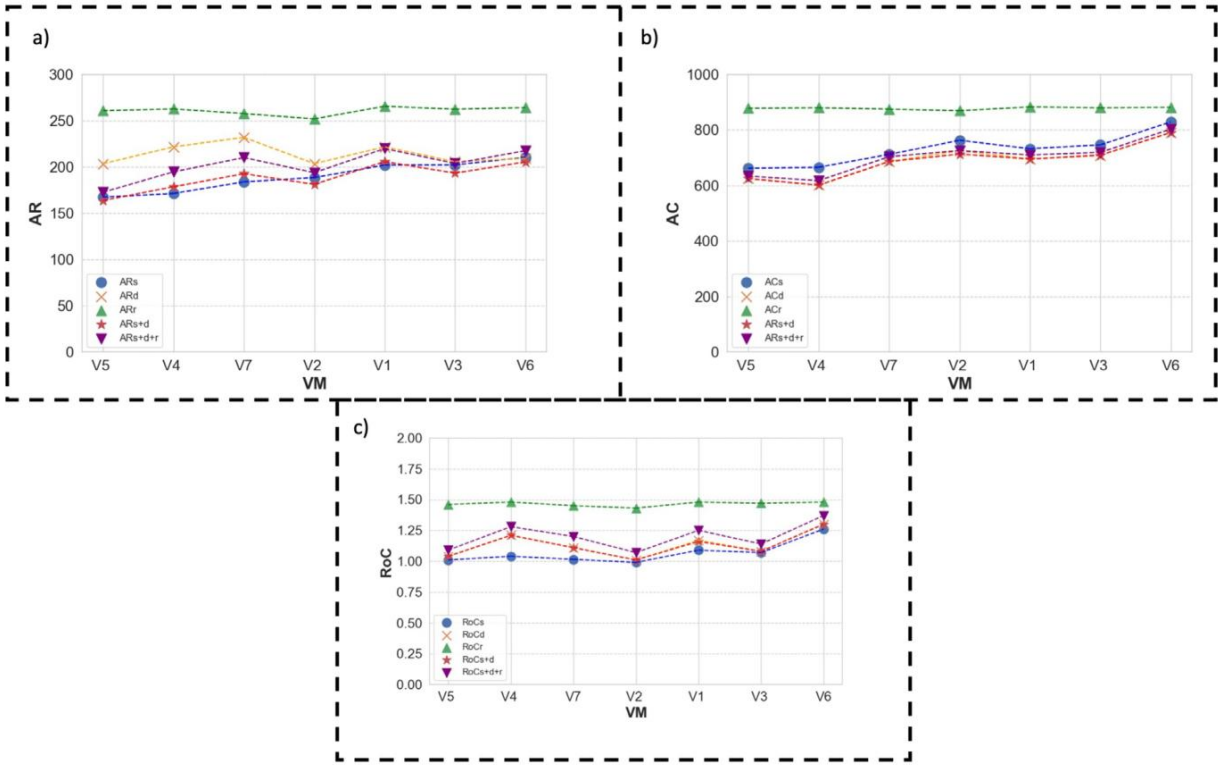
MTD kích hoạt theo thời gian (chu kỳ 10 phút)

- Shuffle: di trú máy ảo, đổi IP qua SDN
- Diversity: đổi hệ điều hành
- Redundancy: nhân bản máy ảo

Chỉ số đánh giá

- ASP (xác suất tấn công thành công), rủi ro AR, chi phí tấn công AC, RoC = R/AC
- Độ trung tâm của đồ thị để chọn máy ảo cần bảo vệ

Thực nghiệm, kết quả và hạn chế của bài báo



Hình 6 của bài báo: AR, AC, RoC sau khi áp MTD (s, d, r, s+d, s+d+r) lên từng máy ảo

Thực nghiệm

Mô phỏng 4 host, 8 máy ảo (Linux, Windows 11, FreeBSD), 18 CVE hệ điều hành + 14 CVE dịch vụ kèm CVSS; không dùng dataset.

Kết quả

Shuffle + diversity cho rủi ro thấp nhất, chi phí tấn công cao hơn. Ví dụ V5: R_s 167.38 · R_d 163.73 · R_r 260.84.

Hạn chế

Chỉ mô phỏng; xác suất khai thác tĩnh; chỉ MTD theo thời gian, không có bộ phát hiện kích hoạt; không công bố mã.

→ Đề tài: Track A phát hiện botnet bằng autoencoder (theo Meidan 2018) · Track B tái lập THARM và MTD lai do cảnh báo AE kích hoạt

Dữ liệu N-BaIoT: lớn, trùng lặp và mất cân bằng

7.06M

dòng (UCI, CC BY 4.0)

115

đặc trưng = 23 thống kê × 5 cửa sổ

9

thiết bị IoT thương mại

10

kiểu tấn công Mirai + BASHLITE

157,779

dòng trùng lặp chính xác bị loại (benign Danmini 18.5%; gafgyt tcp/udp). Còn 6,904,827 dòng; manifest SHA-256 cho 107 file.

5.5–61.8×

attack nhiều hơn benign → Accuracy vô nghĩa; metric chính là macro-F1 và FPR mức cửa sổ, kèm F1 khi test cân bằng.

Chống rò rỉ: chia theo thời gian, có vùng đệm

DStrn · train AE + fit scaler | **DSopt · ngưỡng tr*, cửa sổ ws*** | **Test benign + mọi tấn công**

← thứ tự thời gian của benign (1/3 · 1/3 · 1/3 như Meidan); khoảng trắng giữa các khối = 500 dòng đệm bị bỏ →

0.20–0.86

tự tương quan bậc 1 của benign theo thứ tự gốc (xáo trộn: ≈ 0.00) → chia ngẫu nhiên sẽ rò rỉ

- Khử trùng lặp trong từng file
- Scaler fit chỉ trên DStrn, gắn tên thiết bị
- Ngưỡng/cửa sổ chỉ từ DSopt benign
- Vote không vượt ranh giới file
- Đối chứng E6: cố tình làm sai để đo sai lệch

Pipeline và các mô hình so sánh



Mô hình	Kiến trúc	Tham số	Vai trò
B0	115-86-58-38-29-…-115	36,876	Baseline Meidan, per-device
B1	B0 dùng chung 9 thiết bị	36,876	Kiểm tra giả thuyết per-device
B2	Isolation Forest	–	Baseline không phải NN
V1	115-32-8-32-115	8,059	AE nhẹ (21.9% tham số)
V2	23-12-4-12-23 × 5 cửa sổ	~700	Bớt cả đặc trưng

Hệ thống demo tách 4 tầng: AI · BE · DB · FE



AI

ai/ – huấn luyện, đánh giá, lượng tử hóa, benchmark, xuất gói triển khai



Backend

FastAPI – nạp gói (kiểm SHA-256), suy luận TFLite 1 luồng, API detect/replay/alerts/MTD



Database

SQLite – devices, models, detection_batches, alerts, mtd_actions, experiment_results

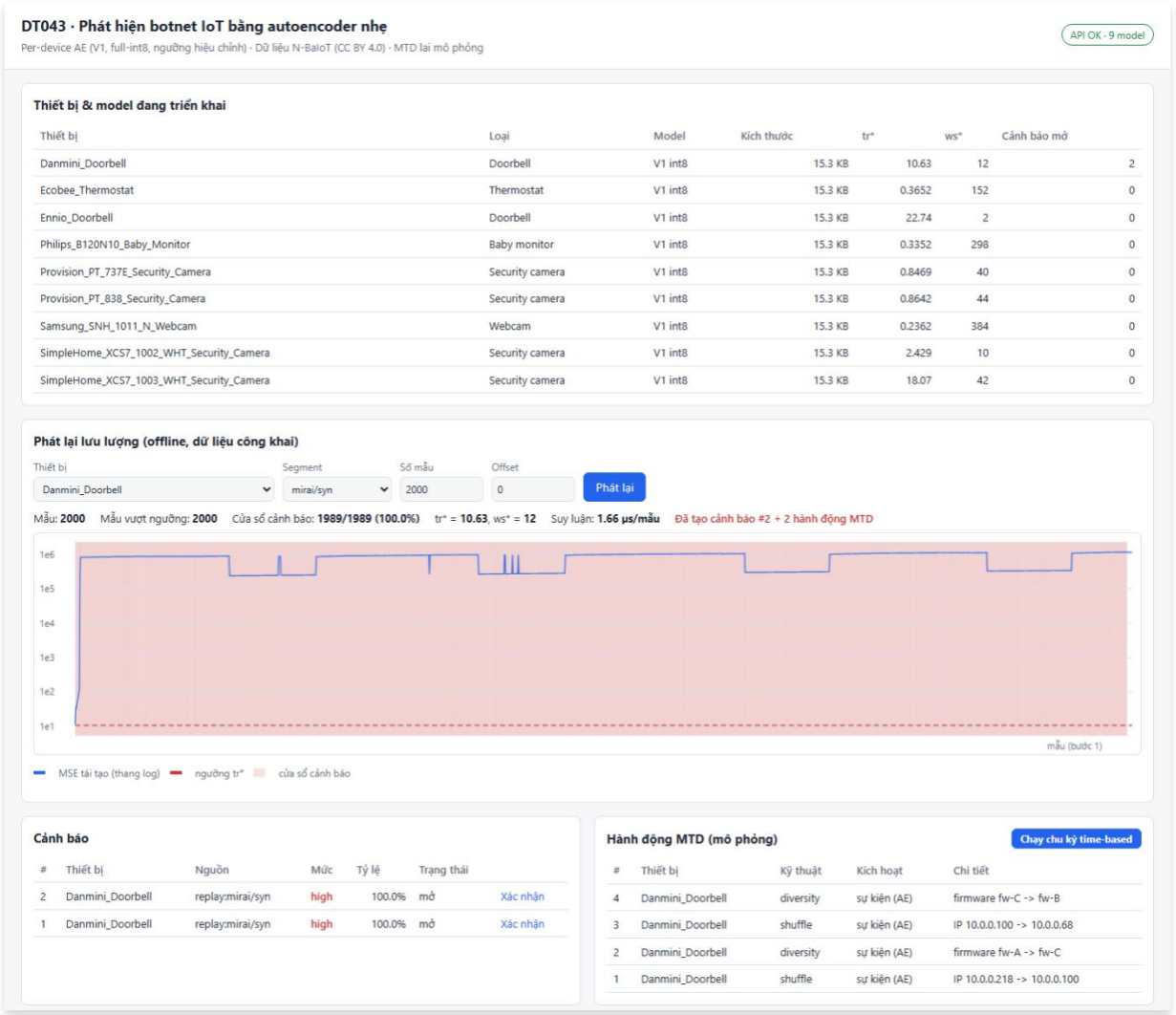


Frontend

Dashboard HTML/JS – thiết bị, phát lại lưu lượng, biểu đồ MSE, cảnh báo, nhật ký MTD

Hợp đồng AI → BE: artifacts/deploy/<thiết bị>/{model.tflite, scaler.json, meta.json} · 56 test tự động (44 AI + 12 BE/DB)

Demo: phát hiện + cảnh báo + MTD trên dashboard



mirai/syn (Danmini)

100% cửa sổ cảnh báo → cảnh báo mức high + shuffle IP + đổi firmware (event-based)

benign (phần test)

Không cảnh báo, không move vô ích

gafgyt/tcp (Ennio)

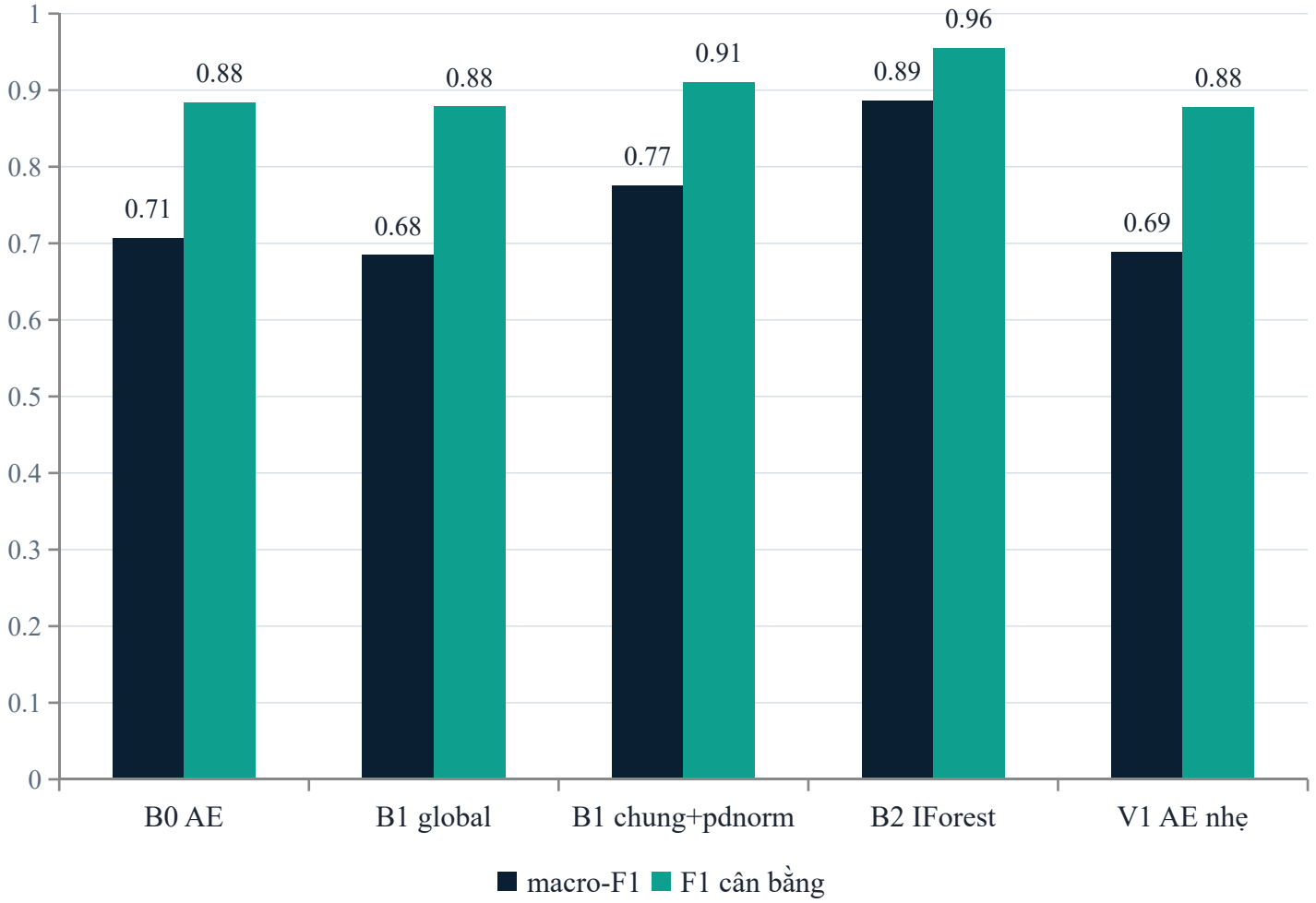
Không vượt ngưỡng – đúng lỗi bỏ sót đã phân tích

Suy luận V1 int8

Vài µs/mẫu, 1 luồng; mọi lô ghi vào SQLite

Mở: http://127.0.0.1:8000/?replay=Danmini_Doorbell:mirai/syn

Kết quả tổng quan (mức cửa sổ, 9 thiết bị × seed)



0.0025

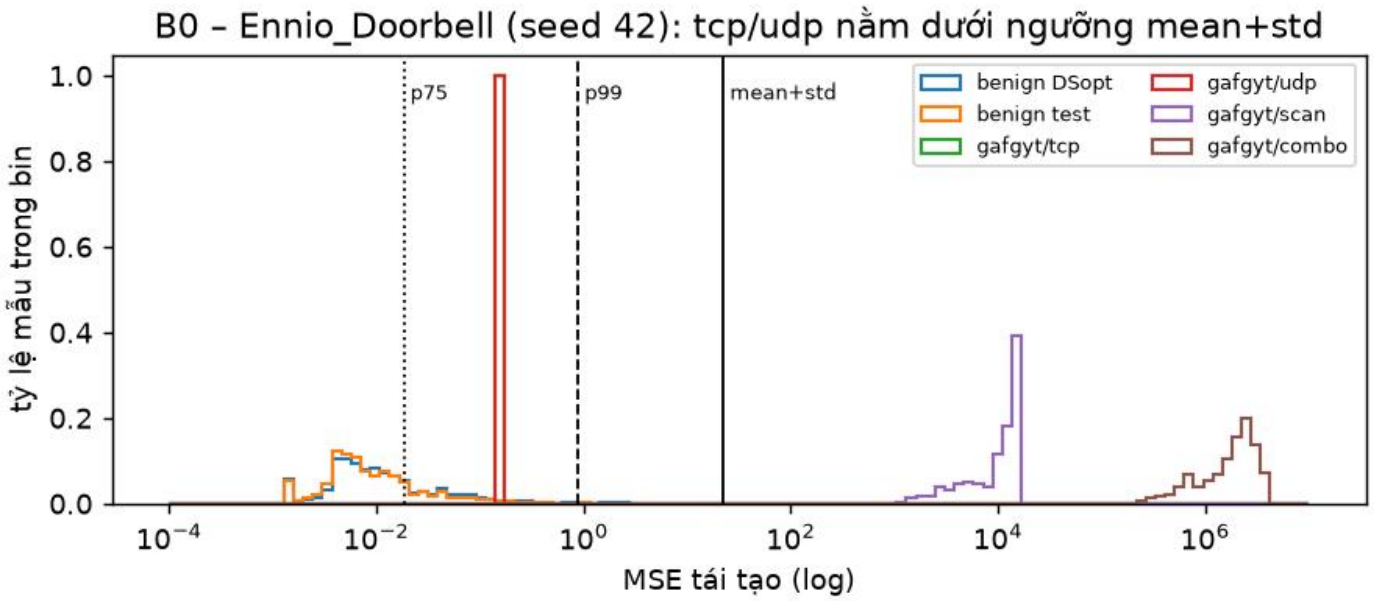
FPR của B0 – thấp hơn Meidan (0.007)

0.822

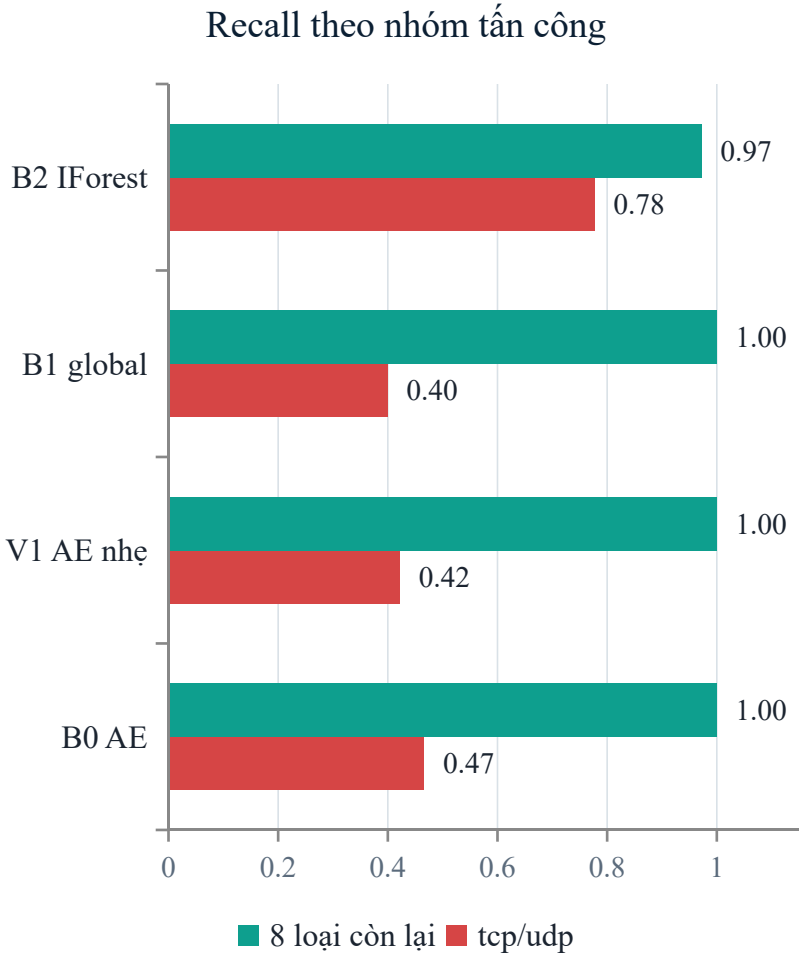
TPR của B0 – không đạt 100% như paper

ROC-AUC mức mẫu vẫn > 0.99: điểm MSE phân tách tốt, vấn đề nằm ở ngưỡng.

Lỗi chính: flood BASHLITE tcp/udp trông giống benign



Nguồn bị giả mạo liên tục → mỗi gói như "luồng mới": weight = 1, variance = 0, 60 byte. MSE ≈ 0.17 – trên phần lớn benign nhưng dưới ngưỡng mean+std bị đuôi phân phối kéo lên.

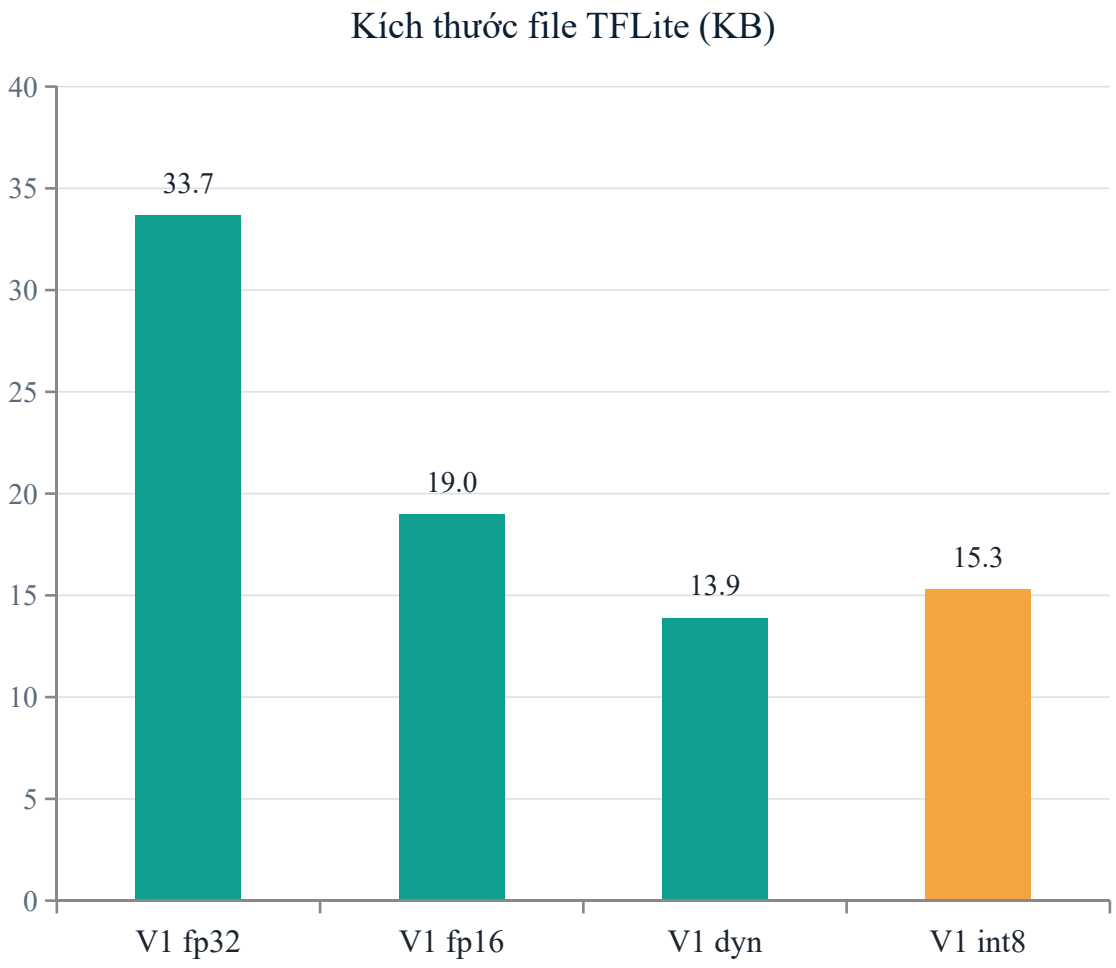


Hậu kiểm: hạ ngưỡng bắt được tcp/udp nhưng phải trả giá

Ngưỡng mean+std (gốc)		Ngưỡng p75 của DSopt	
macro-F1	**0.686**	macro-F1	**0.995**
Recall tcp/udp	**0.44**	Recall tcp/udp	**1.00**
FPR test	**0.0023**	FPR test	**0.0203**
Cửa sổ ws*	**164 mẫu**	Cửa sổ ws*	**1410 mẫu**

Phân tích khám phá làm SAU khi xem test (V1, seed 42) → chỉ để giải thích, không tuyên bố cải tiến. FPR = 0 trên DSopt nhưng tăng trên test: benign trôi theo thời gian.

Lượng tử hóa: nhỏ hơn, và cần hiệu chỉnh lại ngưỡng



int8: giữ ngưỡng fp32 → hiệu chỉnh trên DSopt

FPR	**0.0030 → 0.0016**
Cửa sổ ws*	**153 → 113**
macro-F1	**0.751 → 0.717**

Hiệu chỉnh giảm FPR và độ trễ, nhưng macro-F1 giảm: khi giữ ngưỡng cũ, nhiễu int8 "vô tình" đẩy tcp/udp vượt ngưỡng.

Edge benchmark: V1 int8 trên một lõi E-core

10.8 µs

p95 latency / mẫu (ngưỡng 1000 µs)

15.3 KB

file model (ngưỡng 50 KB)

4.6 MB

RAM tăng thêm khi nạp + chạy

102,899

mẫu/giây trên 1 lõi

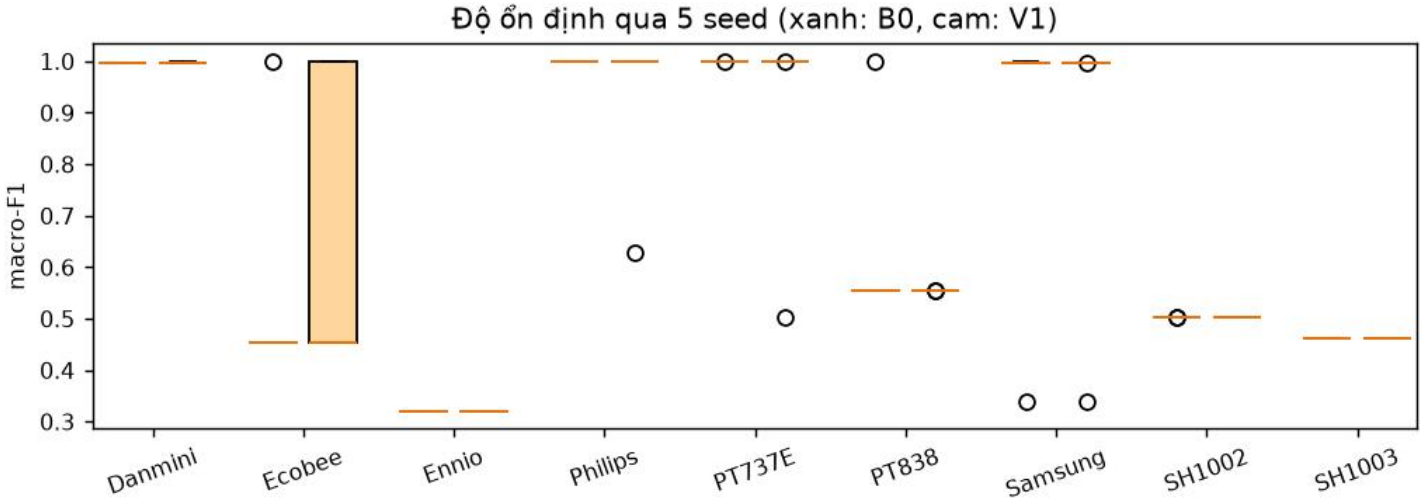
Đa luồng làm chậm model nhỏ

Batch 1 với 20 luồng: p50 473 µs, so với 9.0 µs khi 1 luồng ghim E-core → trên edge nên chạy đơn luồng.

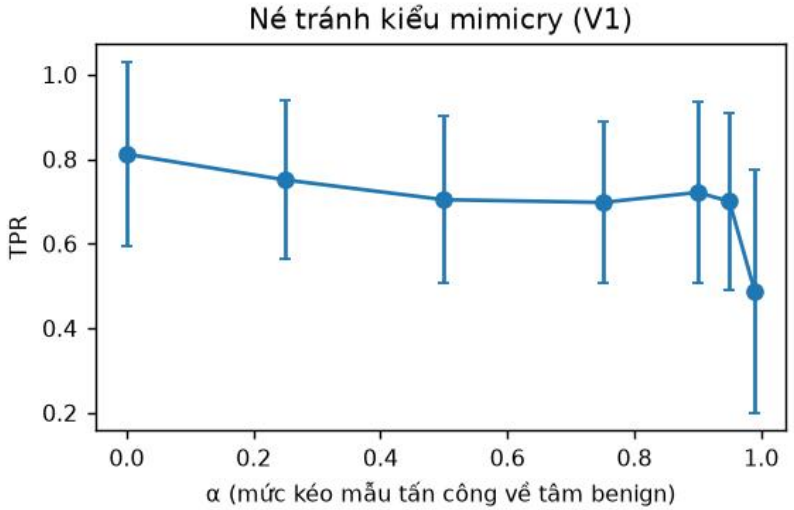
Energy proxy & giới hạn

≈ 40.5 µJ/mẫu (CPU-time × 55 W/14 lõi), 7,872 MAC/mẫu. Không có Raspberry Pi: lõi E-core laptop mạnh hơn CPU edge thật.

Độ ổn định theo seed và robustness



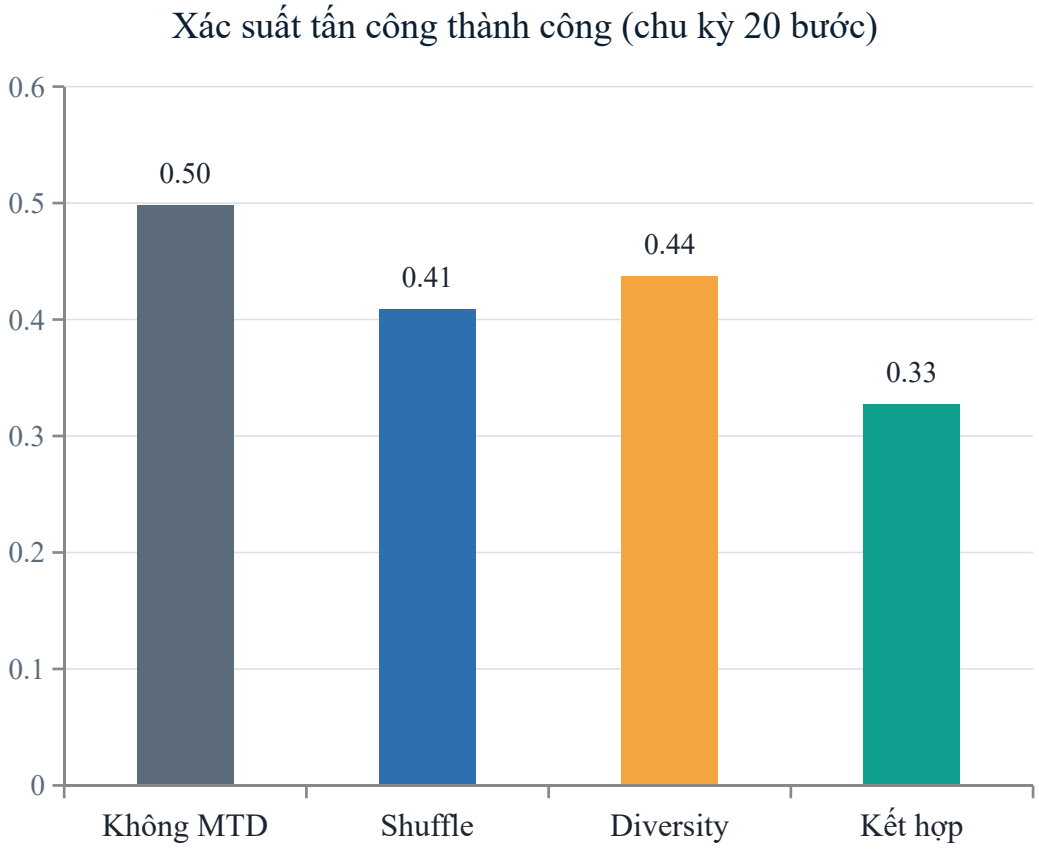
7 cặp (model, thiết bị) có TPR dao động > 0.05 chỉ vì đổi seed – một lần chạy như Meidan là không đủ.



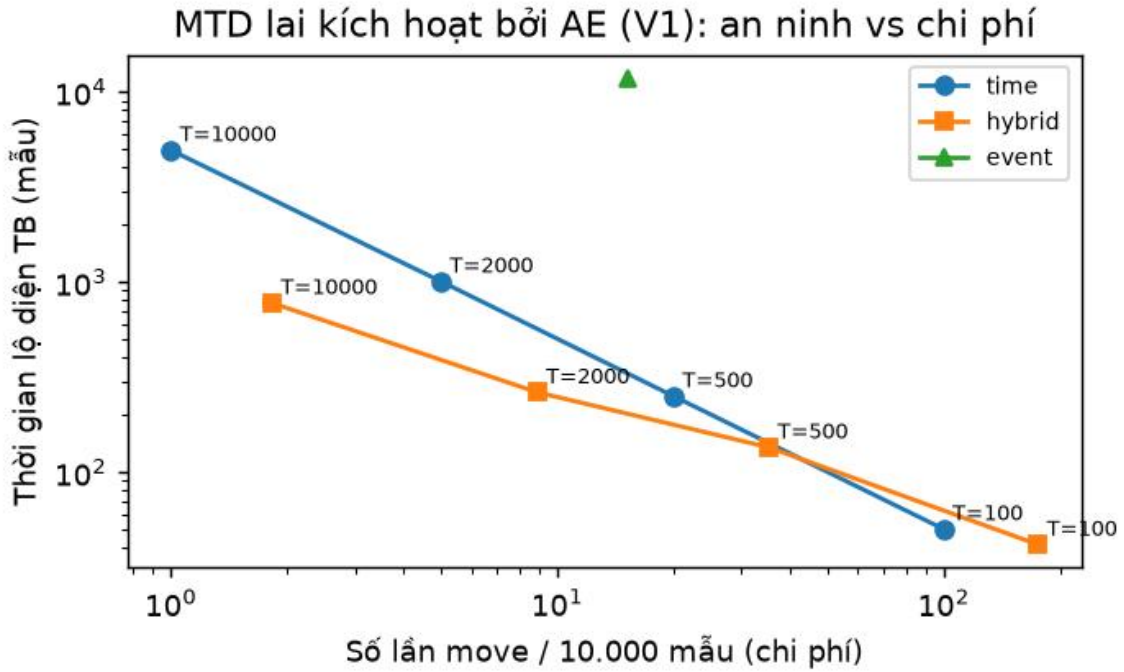
Né tránh: kéo mẫu tấn công về tâm benign; phải bắt chước rất sát (α lớn) mới né được – tức là phải giảm cường độ tấn công.

Báo cáo mean ± std, KTC95 và Wilcoxon thay vì con số tốt nhất.

MTD: tái lập xu hướng và MTD lai kích hoạt bởi AE



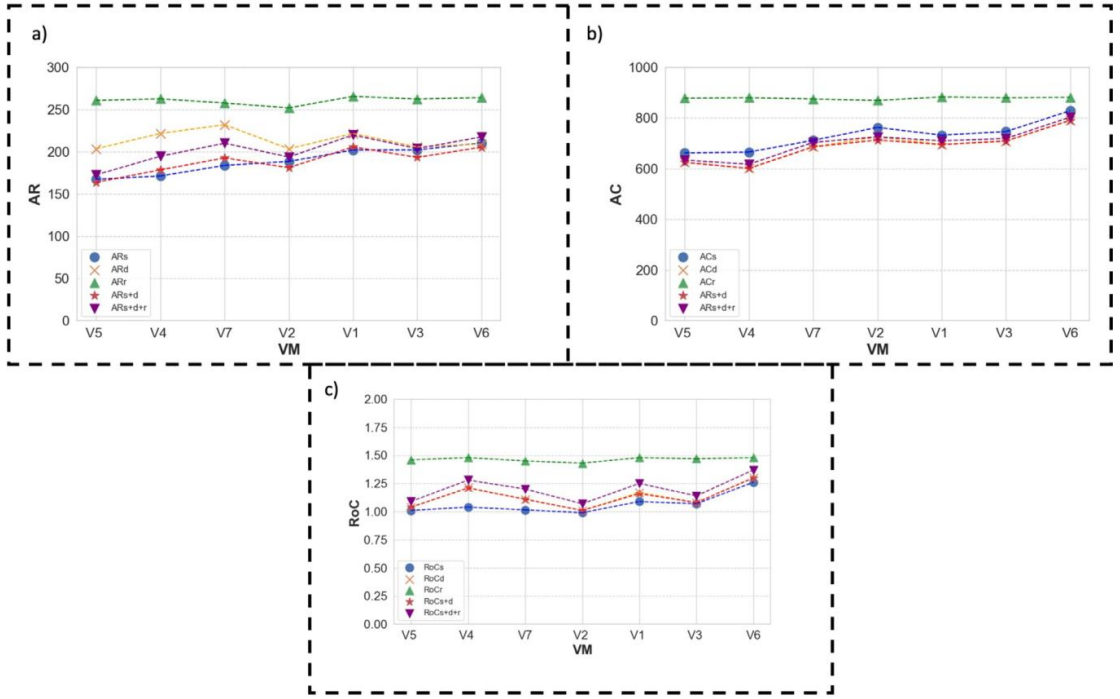
Khớp xu hướng paper chính: kết hợp shuffle + diversity thấp nhất.



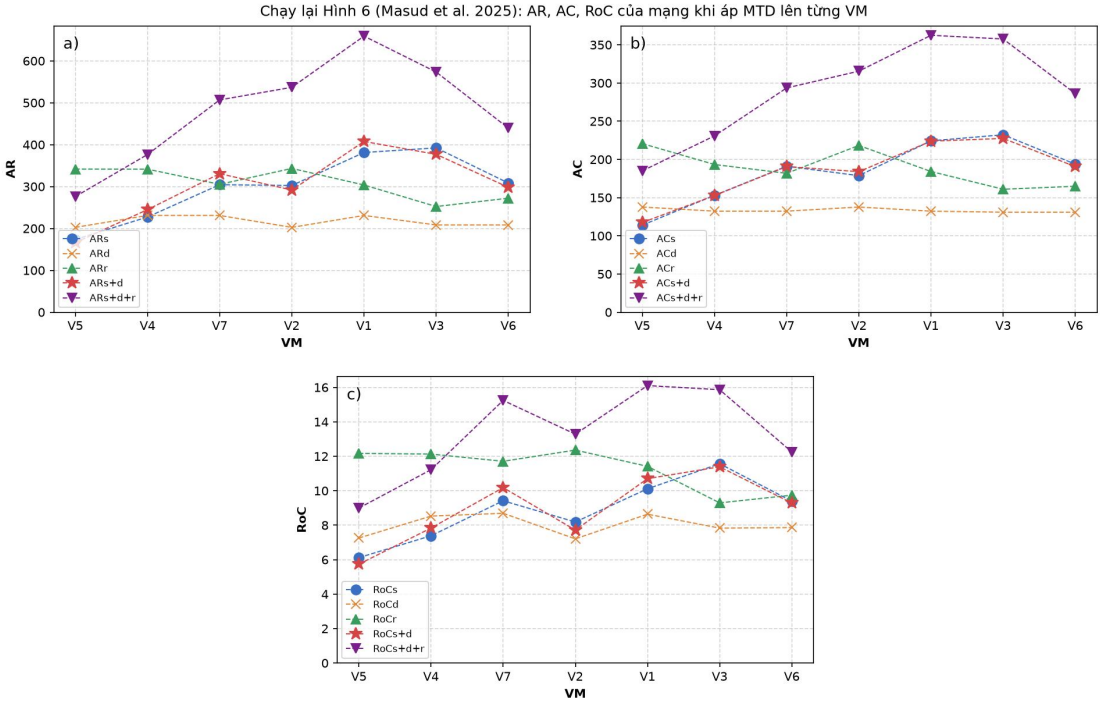
Cùng chi phí, hybrid lộ diện ít hơn time-based. Chỉ event-based: lộ diện TB 11,835 mẫu vì tấn công AE bỏ sót không kích hoạt move → time-based là lưới an toàn.

Chạy lại Hình 6 của bài báo: AR, AC, RoC theo VM

Hình 6 – bài báo



Chạy lại – DT043 (dữ liệu bài báo, topology Hình 3)



Diversity: khớp 7/7 VM (lệch ≤ 5%)

Shuffle, S+D: khớp ở V5 – VM duy nhất bài nêu host đích

Redundancy, S+D+R cao hơn; AC, RoC khác thang đo – bài không nêu đủ cách tính

Đối chiếu tiêu chí đã chốt trước

RQ	Tiêu chí	Giá trị đo	Kết luận
RQ1	TPR ≥ 0.99 và FPR ≤ 0.02 (B0)	TPR=0.8222, FPR=0.0025	**Không đạt**
RQ2	B0 > B1 ở ≥7/9 thiết bị và $p < 0.05$	thắng 4/9, p=0.2852	**Không đạt**
RQ3	V1 ≤ 25% tham số và Δmacro-F1 ≤ 0.01	tham số 21.9%, Δ=+0.0171	**Không đạt**
RQ4	int8+recal: Δmacro-F1 ≤ 0.005 và kích thước ≤ 30% fp32 (V1)	Δ=-0.0278, kích thước 45.5%	**Không đạt**
RQ5	P1: p95 ≤ 1 ms, model ≤ 50 KB, RSS tăng ≤ 50 MB (V1 int8)	p95=10.8 µs, 15.3 KB, ΔRSS=4.6 MB	**Đạt**

Không chỉnh tiêu chí sau khi có kết quả; mọi lần chạy (468) đều được báo cáo.

Kết luận & hướng phát triển

- AE per-device bắt gần tuyệt đối 8/10 loại tấn công với FPR rất thấp; bỏ sót flood tcp/udp giả mạo nguồn
- V1 int8 < 20 KB, vài µs/mẫu trên 1 lõi E-core – khả thi cho edge
- Kết quả dao động theo seed → cần nhiều lần chạy và kiểm định
- MTD: kết hợp shuffle + diversity tốt nhất; MTD lai dùng cảnh báo AE giảm thời gian lộ diện
- Tiếp theo: chọn ngưỡng có kiểm định trên dữ liệu mới · Raspberry Pi + đo điện thật · đặc trưng cho flood giả mạo

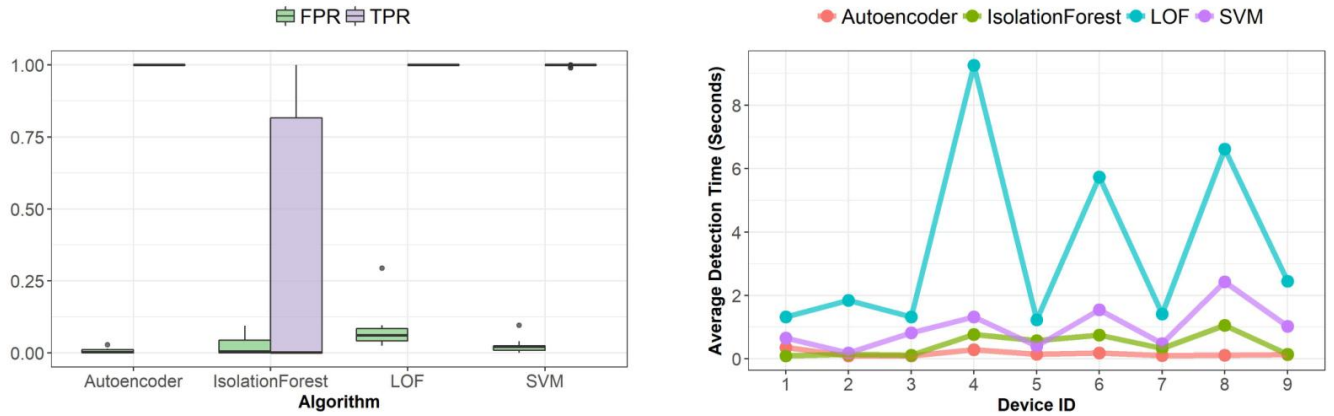
N-BaIoT (Meidan et al., 2018): bài toán, phương pháp, kết quả

Bài toán — Phát hiện thiết bị IoT đã bị nhiễm botnet ngay khi bắt đầu tấn công, chỉ từ lưu lượng mạng.

Phương pháp — Lây nhiễm Mirai, BASHLITE vào 9 thiết bị thật; 115 đặc trưng thống kê; mỗi thiết bị một deep AE học benign; tr* = mean + std, majority vote trên cửa sổ ws*.

Kết quả — TPR 100%, FPR 0.007 ± 0.01 – tốt hơn Isolation Forest, LOF, SVM; phát hiện trong 174 ± 212 ms.

Hạn chế — Chia dữ liệu một lần, không khoảng tin cậy; ngưỡng/cửa sổ chọn để FPR = 0; không lượng tử hóa, không đo trên edge.



(a) Methods' detection accuracy (b) Methods' detection time (seconds)

Hình 2 (a, b) của bài báo: TPR/FPR và thời gian phát hiện của autoencoder so với Isolation Forest, LOF, SVM

Quy trình thực hiện: từ đọc bài báo đến báo cáo

1 Đọc bài báo

Masud 2025 lệch chủ đề (MTD, không AE) → chia 2 track; Meidan 2018 làm baseline

2 Lập kế hoạch

Phiếu tái lập, ≥ 5 hạn chế, 6 câu hỏi nghiên cứu, ma trận thí nghiệm

3 Môi trường, dữ liệu

Python 3.11 + khóa phiên bản; tải N-BaIoT 1.77 GB, manifest SHA-256 107 file

4 Làm sạch, EDA

Kiểm schema 115 cột, khử 157,779 dòng trùng, đo mất cân bằng, tự tương quan

5 Pipeline + test

Chia chống rò rỉ, scaler, AE, ngưỡng, int8; 30 test unit / integration / negative

6 Chốt tiêu chí

Smoke test 1 lần → chốt tiêu chí thành công, lưu SHA-256 trước khi chạy

7 Chạy ma trận

468 lần chạy (cấu hình × thiết bị × seed), mỗi lần 1 process, có resume và log

8 Hệ thống 4 tầng

Xuất gói AI → BE; FastAPI + SQLite + dashboard; 12 test BE/DB

9 Tổng hợp, báo cáo

KTC95, Wilcoxon + Holm, đối chiếu tiêu chí; Word và slide sinh từ số liệu

10 Tái lập bài chính

Có toàn văn: dựng 3-layer THARM bằng đúng CVE/topology (E11), 14 test so số

Mỗi bước có lệnh đã chạy, kết quả và sự cố gặp phải trong tailieu/NHAT_KY_THUC_HIEN.md; bước 6 làm trước khi có kết quả nên không thể chỉnh tiêu chí theo kết quả.

Phương pháp phát hiện: chi tiết từng bước

1. Chia benign của mỗi thiết bị theo thời gian: DStrn | DSopt | test (mỗi phần 1/3), bỏ 500 dòng đệm ở mỗi ranh giới; dữ liệu tấn công chỉ ở test.
2. Chuẩn hóa z-score, fit chỉ trên DStrn của đúng thiết bị (scaler gắn tên thiết bị).
3. Huấn luyện AE chỉ trên benign: Adam lr 1e-3, batch 256, early stopping (patience 8) theo DSopt, tối đa 100 epoch.
4. Điểm bất thường của mỗi mẫu = MSE tái tạo (tính bằng float64 vì có đặc trưng tới 5.9e17).
5. Ngưỡng tr* và cửa sổ ws* chỉ lấy từ DSopt benign (công thức bên phải).
6. Lượng tử hóa TFLite (fp16, dynamic, full-int8 với dữ liệu đại diện từ DStrn), rồi tính lại tr*, ws* bằng chính model int8.
7. Đánh giá trên test ở mức cửa sổ: macro-F1, FPR, TPR; 5 seed, KTC95, Wilcoxon + Holm trên 9 thiết bị.

Công thức

$$MSE(x) = (1/115) \cdot \Sigma (x_i - \hat{x_i})^2$$

tr* = mean(MSE_DSopt) + std(MSE_DSopt)

Cửa sổ ws* mẫu bị cảnh báo khi hơn một nửa số mẫu có MSE > tr*

ws* = cửa sổ nhỏ nhất cho FPR = 0 trên DSopt

Vì sao hiệu chỉnh sau int8?
Lượng tử hóa làm lệch phân phối MSE; ngưỡng chỉ dùng DSopt benign nên tính lại được mà không chạm vào test.

Thực nghiệm: ma trận E1–E11 và môi trường chạy

ID	Nội dung	Số lần chạy	Phục vụ
E1	B0 – tái lập baseline Meidan	9 × 5	RQ1
E2	B1 global; B1 chung + chuẩn hóa riêng	9 × 5 × 2	RQ2
E3	B2 Isolation Forest	9 × 5	baseline
E4	V1 nhẹ; V2 × 5 cửa sổ	45 + 135	RQ3
E5	fp32 / fp16 / dyn / int8 × giữ / hiệu chỉnh ngưỡng	trong E1, E4	RQ4
E6	Đối chứng rò rỉ (cố tình chia ngẫu nhiên)	9 × 3	chống rò rỉ
E7	Ablation 4 quy tắc ngưỡng × 3 cửa sổ	trong E1, E4	ablation
E8	Edge benchmark P0 / P1	3 × 4 × 3 × 5 lượt	RQ5
E9	Robustness: né tránh, nhiễu, đầu độc 1% / 5%	9 × 3 × 3	an toàn
E10	MTD: HARM rút gọn + MTD lai do AE kích hoạt	5 seed Monte Carlo	RQ6
E11	Tái lập 3-layer THARM bằng dữ liệu bài báo	tất định, 14 test	bài chính

Môi trường

- i7-13650HX (6 P + 8 E core), RAM 23.7 GB, Windows 11
- Python 3.11.9, TensorFlow 2.21, scikit-learn 1.9
- Seed 42, 1337, 2024, 7, 123

Nguyên tắc

- Mỗi cấu hình là 1 file YAML; lưu config, seed, log, metric từng lần chạy
- Giữ cả lần chạy lỗi; tổng 468 kết quả, không chọn seed đẹp
- Test chỉ dùng để đánh giá cuối cùng

Tái lập 3-layer THARM: lấy gì từ bài báo, tính thế nào

Lấy từ bài báo	Nguồn
18 CVE hệ điều hành: CVSS, exploitability E, impact I, cost	Bảng 1
14 CVE của dịch vụ s1–s14	Bảng 2
Máy ảo → hệ điều hành (Linux, Windows 11, FreeBSD)	Hình 1
Máy ảo → dịch vụ; V8 là CSDL đích	Hình 3
Đồ thị tấn công giữa máy ảo (11 cạnh)	Definition 2
Shuffle: V5 di trú host 2 → host 4	Mục 4.2
Diversity: đổi hệ điều hành dự phòng	Mục 4.3
Redundancy: nhân bản V5 → V5′	Mục 4.4

1 — Dựng 3 tầng

Cây tấn công OR cho từng máy ảo → đồ thị dịch vụ → đồ thị máy ảo, không tự đặt tham số.

2 — Rủi ro máy ảo

$R_v = \Sigma E \cdot I$. Bản in ghi tích ($\prod$) nhưng chỉ dạng tổng mới khớp các số bài báo dùng.

3 — Đường tấn công

Rủi ro đường = tổng R các máy ảo trên đường; AC = Σ cost; ASP theo OR độc lập.

4 — So với bài báo

Áp shuffle / diversity / redundancy và tổ hợp; so số công bố và 7 kết luận định tính trên 2 topology.

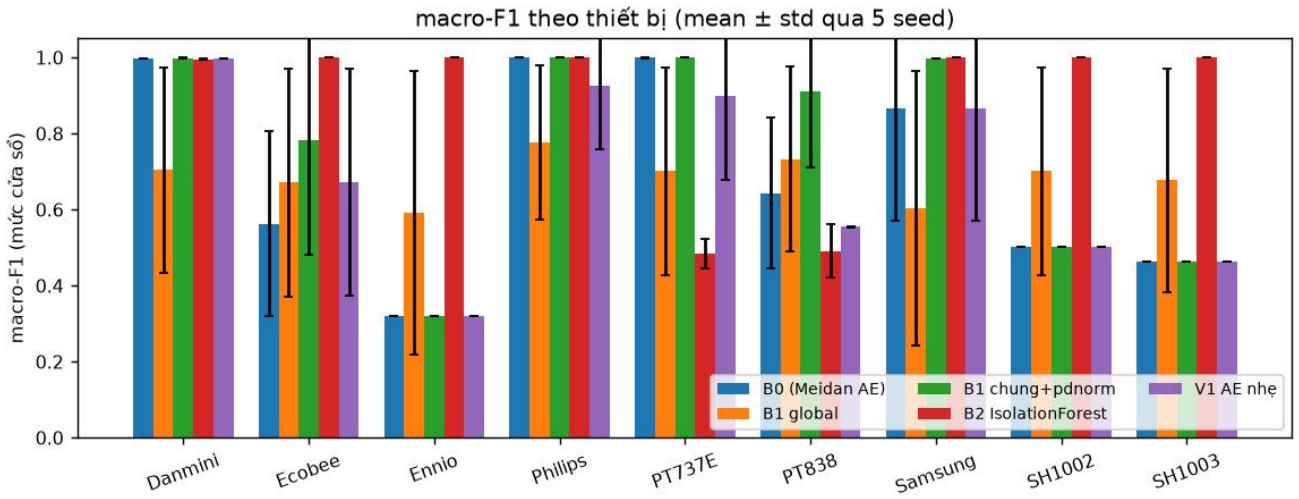
Mã: ai/src/nbaiot/mtd/tharm.py · script ai/scripts/10_tharm_paper_replication.py · 14 test khóa các số của bài báo · chi tiết: tailieu/docs/tai_lap_paper_chinh.md

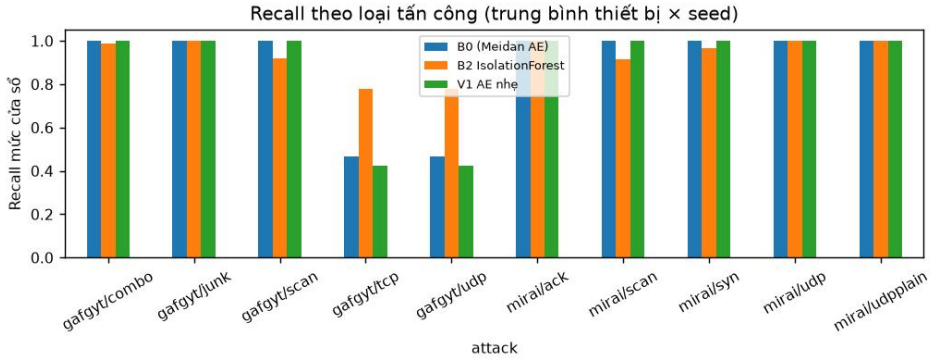
Kết quả tái lập Meidan 2018 trên N-BaIoT

Chỉ số	Meidan 2018	DT043 B0	V1 int8	Nhận xét
TPR	100%	0.822	0.829	Không đạt
FPR	0.007	0.0025	0.0016	Thấp hơn
ws* (mẫu)	20–82	152	113	Dài hơn
Model	172 kB	147.7 KB	15.3 KB	Nhỏ ~10×
Chia dữ liệu	1 lần	5 seed	5 seed	Chặt hơn

Vì sao lệch so với bài báo

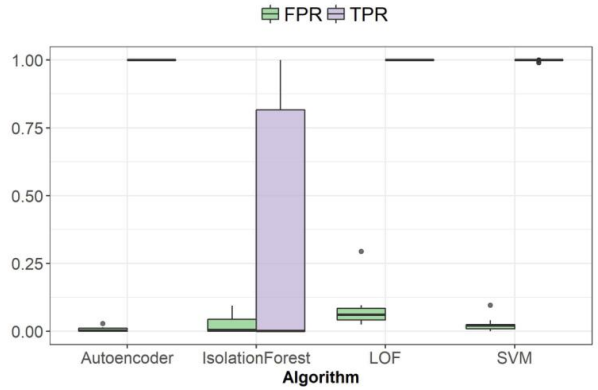
- Flood tcp/udp của BASHLITE giả mạo nguồn → giống benign, MSE dưới ngưỡng
- Đuôi MSE trên DSopt kéo ngưỡng mean + std lên cao
- Khử trùng lặp + vùng đệm → ws* dài hơn
- 8/10 loại tấn công còn lại recall ≈ 1



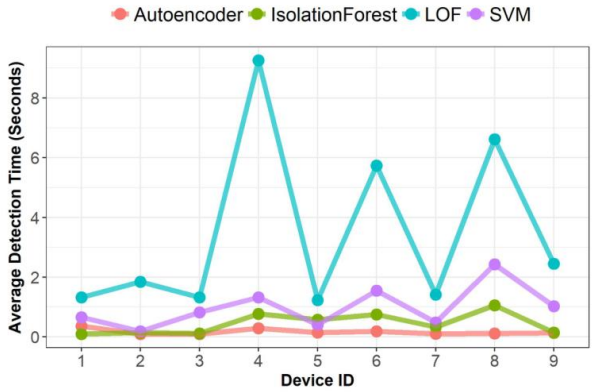


Chạy lại Hình 2 (a, b) của Meidan 2018

Hình 2 – bài báo

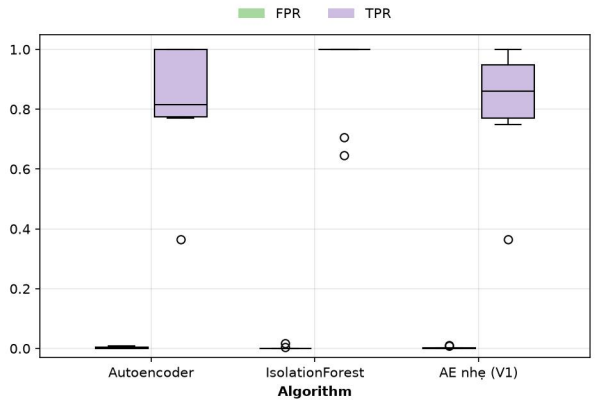


(a) Methods′ detection accuracy

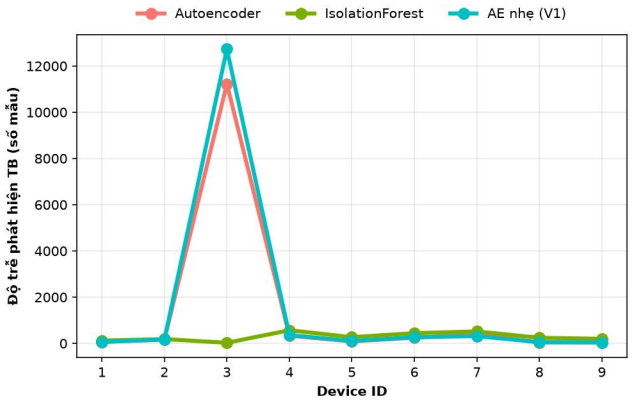


(b) Methods′ detection time (seconds)

Chạy lại – DT043 (468 lần chạy, trung bình theo seed)



(a) Độ chính xác phát hiện (9 thiết bị)



(b) Độ trễ phát hiện theo thiết bị (đo bằng số mẫu)

(a) Độ chính xác

- FPR gần 0 ở mọi thuật toán – khớp bài báo
- TPR của AE: bài 100%, chạy lại trung vị 0.82 do flood tcp/udp giả mạo nguồn
- Isolation Forest chạy lại tốt hơn AE (bài: ngược lại)

(b) Thời gian phát hiện

- Bài đo giây trên lưu lượng thật; N-BaIoT không có timestamp nên đo bằng số mẫu
- Thiết bị 3 (Ennio) trễ lớn vì AE bỏ sót tcp/udp

Chưa chạy lại được

- LOF, One-class SVM (chưa nằm trong ma trận thí nghiệm)
- Hình 2 (c, d): cần lưu lượng theo giờ – không có trong dataset

Kết quả tái lập bài báo chính (Masud 2025)

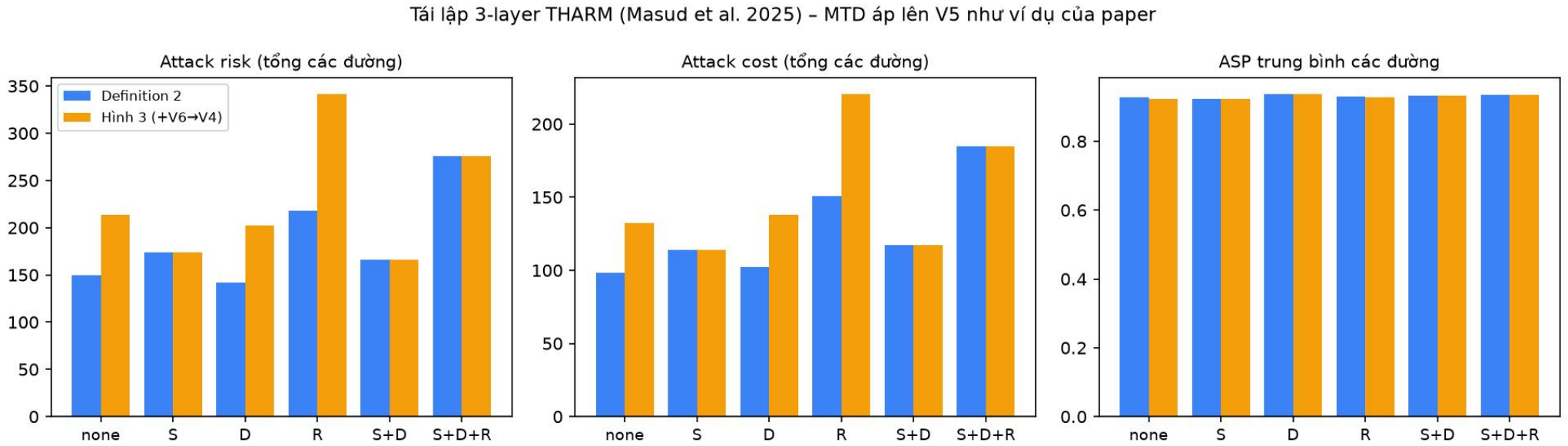
Mức so sánh	Bài báo	Tái lập	Kết luận
Rủi ro V4 · V5 · V7 · V8	12.47 · 12.58 · 8.67 · 5.62	12.47 · 12.58 · 8.67 · 5.63	Khớp ±0.01
3 đường tấn công ví dụ	43.15 · 43.31 · 43.31	42.81 · 42.97 · 42.97	Khớp −0.8%
Tổng mạng R_s · R_d · R_r	167 · 164 · 261	174 · 142 · 218	Không khớp

6/7

kết luận khớp (Hình 3)

3/7

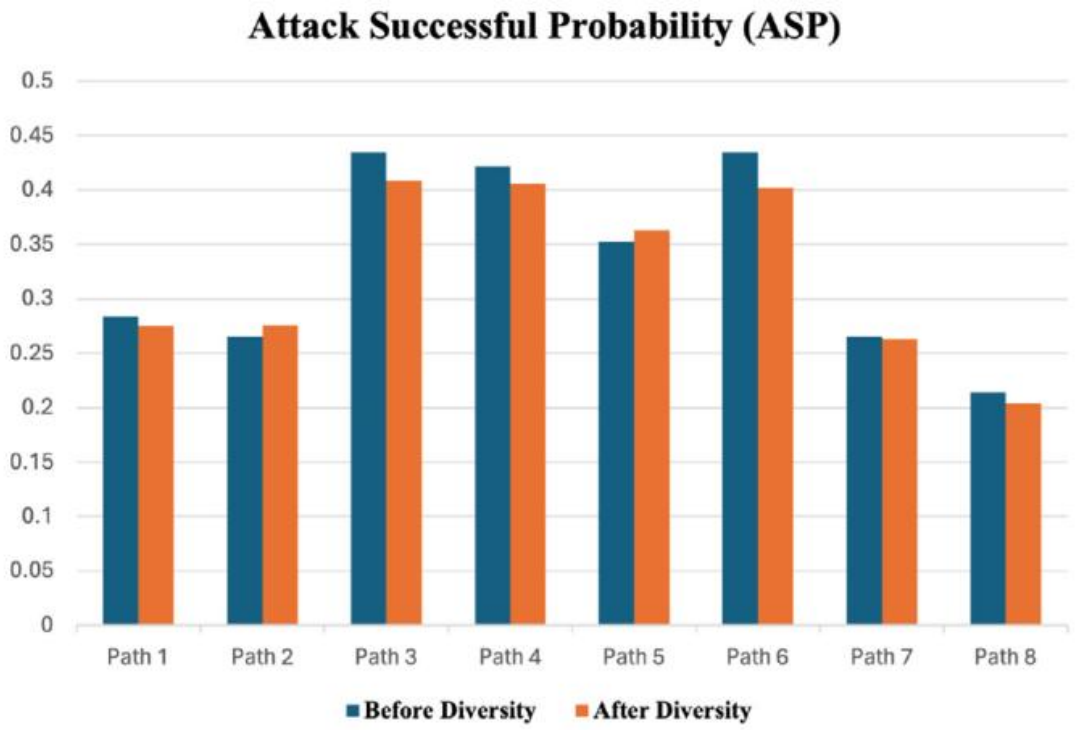
theo Definition 2



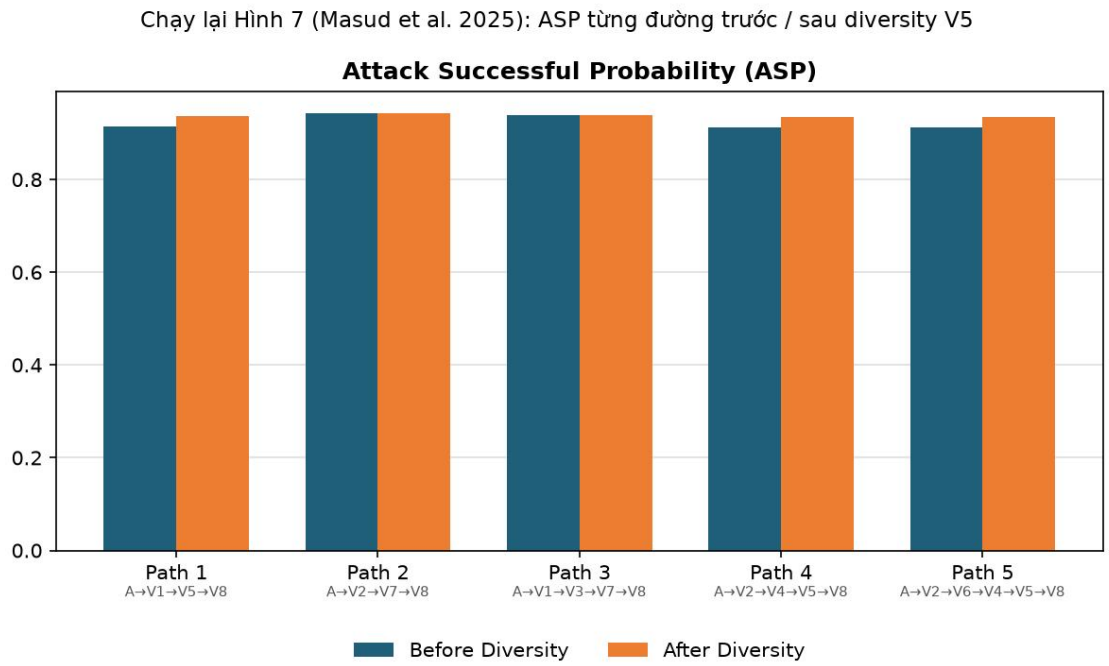
Tổng toàn mạng không suy ra được: bài báo cắt danh sách cạnh sau shuffle, công thức in mâu thuẫn; kết luận "S + D tốt nhất" phụ thuộc cạnh V6 → V4.

Chạy lại Hình 7: ASP trước / sau diversity

Hình 7 – bài báo (8 đường)



Chạy lại – DT043 (5 đường tới V8, diversity V5)



Không tái lập được: topology bài công bố chỉ có 5 đường tới V8 (bài vẽ 8 đường); ASP đường của bài 0.2–0.43 trong khi ASP từng VM ở Bảng 4 là 0.95–0.99 – bài không nêu cách tính ASP đường. Với OR độc lập, đổi V5 Windows 11 → Linux làm ASP tăng nhẹ.

Nguyên mẫu: luồng dữ liệu, gói triển khai và API



Gói triển khai mỗi thiết bị (artifacts/deploy/<thiết bị>/)

- model.tflite – V1 int8, 15,688 byte
- scaler.json – mean/std 115 cột, gắn tên thiết bị
- meta.json – tr*, ws*, danh sách cột, SHA-256 của model
- Backend từ chối nạp nếu checksum lệch hoặc sai thiết bị

Cơ sở dữ liệu (6 bảng)

- devices, models, detection_batches, alerts, mtd_actions, experiment_results – không lưu dữ liệu thô

API	Chức năng
GET /api/health, /api/devices	trạng thái, 9 model đã nạp
POST /api/detect	suy luận một lô đặc trưng
POST /api/replay/{device}	phát lại dữ liệu test công khai
GET /api/alerts · POST …/ack	xem, xác nhận cảnh báo
GET /api/mtd/actions · POST /api/mtd/tick	nhật ký MTD, MTD theo thời gian
GET /api/experiments	bảng kết quả thực nghiệm

56 test tự động: 30 AI + 14 tái lập THARM + 12 backend/CSDL (gói bị sửa 1 byte, path traversal, sai số chiều, lô quá lớn)